

Taller 04: Sockets TCP, UDP e Hilos en Java

Luis Enrique Santos
Santiago José Hernández
José D. Medina
Paula Gabriela Losada

Pontificia Universidad Javeriana
Bogotá, Colombia

Mayo 2026

1. Introducción

El presente documento describe el funcionamiento de una aplicación cliente-servidor desarrollada en Java usando sockets de red. El objetivo principal del taller es comprender cómo se comunican dos procesos mediante los protocolos UDP y TCP.

Para el desarrollo se trabajaron cuatro programas principales: un cliente UDP, un servidor UDP, un cliente TCP y un servidor TCP. También se tuvo en cuenta el uso de hilos, los cuales permiten ejecutar tareas de manera concurrente dentro de una aplicación.

El protocolo TCP se caracteriza por ser orientado a conexión, lo que significa que antes de enviar información se establece un enlace entre el cliente y el servidor. Esto permite garantizar que los datos lleguen correctamente, en orden y sin pérdidas. Por esta razón, TCP es utilizado en aplicaciones donde la confiabilidad es fundamental, como la navegación web, el correo electrónico o la transferencia de archivos.

Por otro lado, el protocolo UDP permite enviar mensajes sin necesidad de establecer una conexión previa. Este método es más rápido y simple, aunque no garantiza que los datos lleguen correctamente o en el orden esperado. UDP es comúnmente utilizado en aplicaciones donde la velocidad es más importante que la confiabilidad, como videojuegos en línea, transmisiones de audio, video y sistemas en tiempo real.

Además, en este taller se aborda el uso de hilos como mecanismo para ejecutar tareas de manera concurrente. Los hilos permiten que un servidor pueda atender varios procesos o clientes al mismo tiempo, evitando bloqueos y mejorando el rendimiento de las aplicaciones de red.

Los sockets permiten que dos aplicaciones intercambien información a través

de una red. En Java, para UDP se utilizan las clases `DatagramSocket` y `DatagramPacket`; mientras que para TCP se utilizan `Socket` y `ServerSocket`.

2. Objetivos

2.1. Objetivo general

Implementar y analizar una aplicación cliente-servidor en Java usando sockets TCP y UDP, comparando su funcionamiento y comprendiendo el uso de hilos.

2.2. Objetivos específicos

- Implementar un servidor y un cliente usando UDP.
- Implementar un servidor y un cliente usando TCP.
- Comparar las diferencias principales entre los protocolos TCP y UDP.
- Explicar el funcionamiento de los hilos en Java.
- Realizar pruebas de ejecución y verificar el envío de mensajes.
- Documentar el código y preparar el proyecto para ser subido a un repositorio.

3. Documentación del código

Cada archivo fuente del proyecto debe estar documentado con comentarios explicativos que indiquen su propósito, autores, fecha y funcionamiento principal. Además, se recomienda incluir un membrete al inicio de cada archivo Java con la información del taller, la institución y los integrantes.

Un ejemplo de membrete para los archivos Java es el siguiente:

```
/*
 * Pontificia Universidad Javeriana
 * Taller 04: Sockets TCP, UDP e Hilos en Java
 * Autores:
 * - Luis Enrique Santos
 * - Santiago Jose Hernandez
 * - Jose D. Medina
 * - Paula Gabriela Losada
 * Fecha: Mayo de 2026
 *
 * Descripcion:
 * Este archivo implementa una parte de la comunicacion
 * cliente-servidor usando sockets en Java.
 */
```

4. Sockets UDP

UDP, o *User Datagram Protocol*, es un protocolo de comunicación que permite enviar mensajes llamados datagramas. Este protocolo no necesita establecer una conexión previa entre cliente y servidor, por lo que suele ser más rápido que TCP.

Sin embargo, UDP no garantiza que los mensajes lleguen correctamente, que lleguen en orden o que lleguen una sola vez. Por esta razón se utiliza principalmente en aplicaciones donde la velocidad es más importante que la confiabilidad absoluta.

En este taller, el servidor UDP escucha mensajes en el puerto 6000. El cliente UDP lee mensajes desde teclado, los convierte a bytes y los envía al servidor.

4.1. Servidor UDP

El servidor UDP crea un socket asociado al puerto 6000:

```
socket = new DatagramSocket(6000);
```

Luego crea un arreglo de bytes para almacenar el mensaje recibido:

```
byte[] mensaje_bytes = new byte[256];  
DatagramPacket paquete = new DatagramPacket(mensaje_bytes, 256);
```

La recepción del mensaje se realiza con:

```
socket.receive(paquete);
```

Cuando el servidor recibe un paquete, convierte los bytes en una cadena de texto y muestra el mensaje por pantalla. El ciclo termina cuando el mensaje recibido comienza con la palabra `fin`.

4.2. Cliente UDP

El cliente UDP recibe como argumento la dirección del servidor. Primero crea el socket:

```
socket = new DatagramSocket();
```

Luego obtiene la dirección del servidor:

```
address = InetAddress.getByName(argv[0]);
```

Cada mensaje ingresado por teclado se convierte a bytes:

```
mensaje_bytes = mensaje.getBytes();
```

Después se crea el paquete indicando el mensaje, la dirección del servidor y el puerto:

```
paquete = new DatagramPacket(mensaje_bytes,
                             mensaje.length(), address, 6000);
```

Finalmente, el paquete se envía con:

```
socket.send(paquete);
```

El cliente continúa enviando mensajes hasta que el usuario escribe una palabra que comienza con **fin**.

5. Sockets TCP

TCP, o *Transmission Control Protocol*, es un protocolo orientado a conexión. Esto quiere decir que antes de enviar datos se establece una conexión entre el cliente y el servidor.

A diferencia de UDP, TCP garantiza que los mensajes lleguen correctamente y en el mismo orden en que fueron enviados. Por esta razón se usa en aplicaciones donde la confiabilidad es importante.

En este taller, el servidor TCP escucha conexiones en el puerto 6001. Cuando un cliente se conecta, el servidor recibe los mensajes enviados y los muestra en pantalla.

5.1. Servidor TCP

El servidor TCP crea un socket de servidor en el puerto 6001:

```
socket = new ServerSocket(6001);
```

Luego espera la conexión de un cliente:

```
Socket socket_cli = socket.accept();
```

Después se crea un flujo de entrada para leer los datos enviados por el cliente:

```
DataInputStream in =
    new DataInputStream(socket_cli.getInputStream());
```

Los mensajes se reciben con:

```
mensaje = in.readUTF();
```

Finalmente, el servidor imprime cada mensaje recibido:

```
System.out.println(mensaje);
```

5.2. Cliente TCP

El cliente TCP obtiene la dirección del servidor:

```
address = InetAddress.getByName(argv[0]);
```

Luego establece la conexión TCP con el servidor en el puerto 6001:

```
socket = new Socket(address, 6001);
```

Para enviar información, se crea un flujo de salida:

```
DataOutputStream out =  
    new OutputStream(socket.getOutputStream());
```

Cada mensaje leído desde teclado se envía usando:

```
out.writeUTF(mensaje);
```

El proceso continúa hasta que el usuario escribe un mensaje que comienza con `fin`.

6. Diferencias entre TCP y UDP

Característica	UDP	TCP
Conexión previa	No	Sí
Confiabilidad	No garantizada	Garantizada
Orden de mensajes	No garantizado	Garantizado
Velocidad	Mayor	Menor
Pérdida de paquetes	Posible	Retransmisión automática
Clases en Java	DatagramSocket, DatagramPacket	Socket, ServerSocket
Puerto usado	6000	6001
Uso típico	Streaming, juegos, tiempo real	Web, correo, archivos

7. Hilos

Los hilos permiten que un programa ejecute varias tareas al mismo tiempo o de forma concurrente. En Java, un hilo puede crearse extendiendo la clase `Thread` o implementando la interfaz `Runnable`.

En aplicaciones cliente-servidor, los hilos son importantes porque permiten atender varias conexiones o ejecutar tareas sin bloquear completamente el programa.

7.1. Ejemplo con Thread

```
public class MiHilo extends Thread {  
    public void run() {  
        System.out.println("Ejecutando hilo");  
    }  
}
```

Para iniciar el hilo se usa:

```
MiHilo hilo = new MiHilo();  
hilo.start();
```

7.2. Ejemplo con Runnable

```
public class MiTarea implements Runnable {  
    public void run() {  
        System.out.println("Ejecutando tarea con Runnable");  
    }  
}
```

Para ejecutar la tarea:

```
Thread hilo = new Thread(new MiTarea());  
hilo.start();
```

La diferencia principal es que **Thread** representa directamente un hilo, mientras que **Runnable** representa una tarea que puede ser ejecutada por un hilo. Usar **Runnable** es preferible porque Java no soporta herencia múltiple, lo que permite mayor flexibilidad de diseño.

8. Resultados de las pruebas

Para la comparativa entre protocolos se enviaron **10.000 mensajes** desde el cliente hacia el servidor, registrando métricas de entrega, integridad, orden y rendimiento en ambos protocolos.

Métrica	UDP	TCP
Mensajes esperados	10000	10000
Mensajes recibidos	9397	10000
Mensajes perdidos	603	0
Errores de integridad	0	0
Fuera de orden	12	–
Tasa de entrega (%)	94.0	100.0
Tiempo recepción (s)	0.2	0.1
Throughput (msg/s)	54917.8	163769.6

Los resultados muestran que TCP entregó los 10.000 mensajes sin pérdida ni errores, con un throughput de 163.769,6 msg/s. UDP, en cambio, perdió 603 mensajes (6 %) y entregaron 12 fuera de orden, aunque presentó un throughput de 54.917,8 msg/s. En ambos protocolos la integridad de los datos recibidos fue perfecta.

Prueba funcional	Resultado esperado	Estado
Enviar mensaje UDP	El servidor UDP recibe el mensaje	Correcto
Enviar fin en UDP	El servidor UDP termina	Correcto
Conectar cliente TCP	El servidor TCP acepta la conexión	Correcto
Enviar mensaje TCP	El servidor TCP recibe el mensaje	Correcto
Enviar fin en TCP	La conexión TCP termina	Correcto
Ejecutar hilos	Se muestran mensajes concurrentes	Correcto

9. Resultados de ejecución de hilos

En esta sección se presentan los resultados obtenidos al ejecutar la parte de hilos del taller. Se comparó una ejecución secuencial con una ejecución concurrente usando hilos.

9.1. Resultado de ejecución secuencial

En la ejecución secuencial, la segunda cajera comienza a procesar al segundo cliente únicamente cuando la primera cajera termina. El tiempo total corresponde a la suma de los tiempos de atención de ambos clientes.

```
La cajera Cajera 1 comienza a procesar Cliente 1 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1: 15seg
La cajera Cajera 2 comienza a procesar Cliente 2 en el tiempo: 15seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 16seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 19seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 24seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 25seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 26seg
La cajera Cajera 2 ha terminado de procesar Cliente 2: 26seg
```

El tiempo total de ejecución fue de 26 segundos.

9.2. Resultado de ejecución con hilos

En la ejecución con hilos, las dos cajeras procesan a los clientes de forma concurrente. Ambas avanzan al mismo tiempo sin que una espere a la otra.

```
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1: 15seg
```

El tiempo total de ejecución fue de 15 segundos. En lugar de sumar los tiempos de los dos clientes, el tiempo total se aproxima al tiempo del cliente que más tarda en ser atendido.

9.3. Comparación entre ejecución secuencial y ejecución con hilos

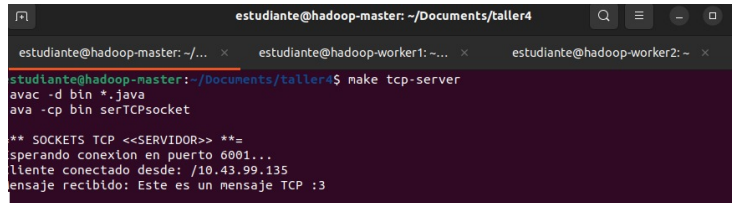
Tipo de ejecución	Comportamiento	Tiempo total
Secuencial	Una cajera después de la otra	26 segundos
Con hilos	Las dos cajeras trabajan al mismo tiempo	15 segundos

La reducción de 26 a 15 segundos demuestra que los hilos aprovechan el tiempo de espera de cada tarea para avanzar con otras, siendo fundamental en servidores que atienden múltiples clientes simultáneamente.

10. Capturas

Las siguientes capturas corresponden a la prueba TCP realizada entre dos máquinas virtuales del cluster: `hadoop-master` como servidor (IP 10.43.100.145) y `hadoop-worker1` como cliente (IP 10.43.99.135).

10.1. Servidor TCP

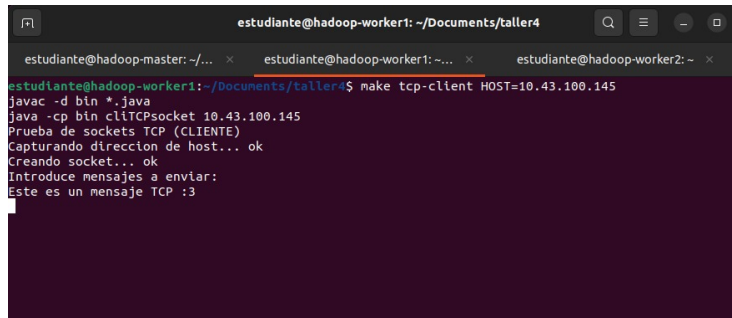


```
estudiante@hadoop-master: ~/Documents/taller4
estudiante@hadoop-master:~/Documents/taller4$ make tcp-server
avac -d bin *.java
ava -cp bin serTCPsocket

** SOCKETS TCP <<SERVIDOR>> **=
esperando conexión en puerto 6001...
cliente conectado desde: /10.43.99.135
mensaje recibido: Este es un mensaje TCP :3
```

Figura 1: Servidor TCP en hadoop-master recibiendo el mensaje desde hadoop-worker1.

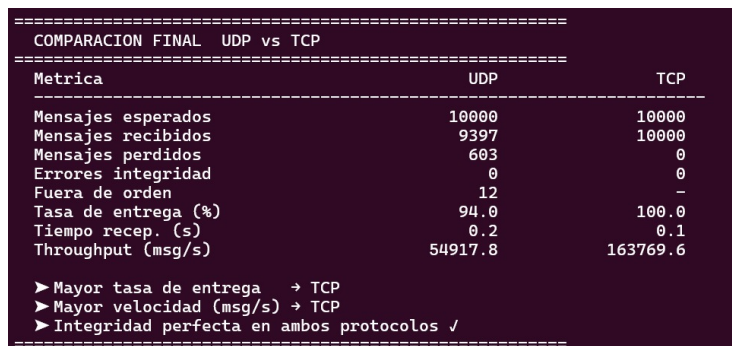
10.2. Cliente TCP



```
estudiante@hadoop-worker1: ~/Documents/taller4
estudiante@hadoop-worker1:~/Documents/taller4$ make tcp-client HOST=10.43.100.145
javac -d bin *.java
java -cp bin cliTCPsocket 10.43.100.145
Prueba de sockets TCP (CLIENTE)
Capturando dirección de host... ok
Creando socket... ok
Introduce mensajes a enviar:
Este es un mensaje TCP :3
```

Figura 2: Cliente TCP en hadoop-worker1 enviando el mensaje al servidor en 10.43.100.145.

10.3. Comparación final UDP vs TCP



COMPARACION FINAL UDP vs TCP		
Metrica	UDP	TCP
Mensajes esperados	10000	10000
Mensajes recibidos	9397	10000
Mensajes perdidos	603	0
Errores integridad	0	0
Fuera de orden	12	-
Tasa de entrega (%)	94.0	100.0
Tiempo recep. (s)	0.2	0.1
Throughput (msg/s)	54917.8	163769.6
➤ Mayor tasa de entrega → TCP		
➤ Mayor velocidad (msg/s) → TCP		
➤ Integridad perfecta en ambos protocolos ✓		

Figura 3: Comparación final entre UDP y TCP tras el envío de 10.000 mensajes: tasa de entrega, throughput y métricas de integridad.

11. Observaciones

- UDP es más simple y rápido porque no establece conexión previa, alcanzando un throughput de 54.917,8 msg/s en la prueba con 10.000 mensajes.
- TCP es más confiable porque garantiza la entrega y el orden de los mensajes. En la prueba entregó los 10.000 mensajes sin ninguna pérdida, con un throughput de 163.769,6 msg/s.
- En UDP se perdieron 603 mensajes (6 %) y 12 llegaron fuera de orden, lo que confirma su naturaleza no confiable.
- En ambos protocolos la integridad de los mensajes recibidos fue perfecta: cero errores de integridad.
- TCP resultó más rápido que UDP en esta prueba (0.1 s vs 0.2 s), lo que se explica por el mayor throughput y la ausencia de retransmisiones necesarias.
- En UDP se trabaja con paquetes o datagramas; en TCP se trabaja con una conexión entre cliente y servidor.
- La prueba TCP entre `hadoop-master` y `hadoop-worker1` validó el funcionamiento en red real y no solo en localhost.
- Los hilos permiten que un programa realice varias tareas sin depender de una ejecución completamente secuencial.
- La palabra `fin` se utilizó como condición para terminar la comunicación.
- Usar `Runnable` es preferible a extender `Thread` porque Java no soporta herencia múltiple.

12. Conclusiones

La implementación realizada permitió comprender cómo funciona la comunicación cliente-servidor mediante sockets en Java. UDP permite enviar mensajes de forma rápida sin establecer una conexión previa, pero no garantiza la entrega de los datos. En la prueba con 10.000 mensajes, UDP perdió 603 paquetes y entregaron 12 fuera de orden, con una tasa de entrega del 94 %.

TCP, por otro lado, establece una conexión entre el cliente y el servidor, garantizando que los mensajes lleguen correctamente y en orden. En la misma prueba, TCP entregó los 10.000 mensajes sin ninguna pérdida ni error, con una tasa de entrega del 100 % y un throughput de 163.769,6 msg/s. La prueba realizada entre `hadoop-master` y `hadoop-worker1` confirmó el funcionamiento correcto en una red real.

También se comprendió la importancia de los hilos en Java, ya que permiten ejecutar tareas de forma concurrente. En la comparación realizada, la

versión secuencial tardó 26 segundos, mientras que la versión concurrente tardó 15 segundos, demostrando que los hilos permiten aprovechar mejor el tiempo de ejecución cuando existen tareas independientes.

Finalmente, se recomienda mantener el repositorio limpio, con código fuente documentado, README claro y sin archivos ejecutables, objetos o comprimidos.

13. Referencias

- Oracle. Documentación oficial de Java: <https://docs.oracle.com/en/java/>
- Oracle. Clase DatagramSocket. <https://docs.oracle.com/javase/8/docs/api/java/net/DatagramSocket.html>
- Oracle. Clase DatagramPacket. <https://docs.oracle.com/javase/8/docs/api/java/net/DatagramPacket.html>
- Oracle. Clase Socket. <https://docs.oracle.com/javase/8/docs/api/java/net/Socket.html>
- Oracle. Clase ServerSocket. <https://docs.oracle.com/javase/8/docs/api/java/net/ServerSocket.html>
- Oracle. Interfaz Runnable. <https://docs.oracle.com/javase/8/docs/api/java/lang/Runnable.html>
- Kurose, J. F. y Ross, K. W. *Computer Networking: A Top-Down Approach*. Pearson.
- Tanenbaum, A. S. y Van Steen, M. *Distributed Systems: Principles and Paradigms*. Prentice Hall.